Naive Bayes classifier

In this chapter, we consider a very simple multi-class classifier that is based on a different principle directly drawn from Bayes' theorem under some strongly simplifying assumptions. Even under such strong assumptions, this classifier is shown to work very well in many practical applications.

As usual, we start by stating our setup. Our input and output variables in our dataset with N samples are:

$$\begin{aligned} \mathbf{x}^{(i)} &= (x_1^{(i)}, \ldots, x_p^{(i)}) \qquad i = 1, \ldots, N \\ y^{(i)} &\in \mathcal{C}_q = \{c_1, \ldots, c_Q\} \end{aligned}$$

i.e., the outcome variable is categorical with Q classes (rather than just two), and $\mathcal{C}_q$ denotes the set of all Q classes.

In general a Bayes classifier uses for prediction the posterior probabilities, i.e. the probabilities of the output class conditional on the data features, using Bayes' theorem:

$$P(Y = c_q \mid X = \mathbf{x}) = \frac{\overbrace{P(X = \mathbf{x} \mid Y = c_q)}^{\spadesuit} \overbrace{P(Y = c_q)}^{\clubsuit}}{\underbrace{P(X = \mathbf{x})}_{\bigstar}}, \quad q = 1, \ldots, Q \tag{4.1}$$

As usual, you can think of the three ingredients of Bayes' theorem as: $\spadesuit$ = 'likelihood', $\clubsuit$ = 'prior' and $\bigstar$ = 'evidence'.

We are now interested in estimating the three expressions ($\spadesuit, \clubsuit$ and $\bigstar$) on the right hand side of equation (4.1) from our data in order to obtain the classifier's estimate. To do so, we will need to make some (strong) assumption - the so-called *'naive assumption'* - hence the name *Naive Bayes*.

Estimation of the components of the classifier (4.1)

'Prior' ($\clubsuit$): Firstly, we turn towards the *prior*, which is labelled $\clubsuit$ in the equation above.

In the absence of any further information, the prior will be given by the frequency of each class in the training data set:

$$P(Y = c_q) = \frac{\sum_{i=1}^{N} I(y^{(i)} = c_q)}{N}, \tag{4.2}$$

where, just as in the previous chapter, $I(\cdot)$ denotes the indicator function:

$$I(b) = \begin{cases} 1, & \text{if } b \text{ is true} \\ 0, & \text{if } b \text{ is false} \end{cases}$$

Of course, if there is additional information in our data set or experimental setup that can be used to inform the prior, then an appropriate prior that incorporates such information could be used instead.

'Likelihood' (♠): Secondly, we consider the second part of Bayes' theorem's numerator, labelled ♠.

Key message: In the likelihood term we make the *naive assumption*, which is to assume the following:

$$P(X = \mathbf{x} \mid Y = c_q) = P(X_1 = x_1, X_2 = x_2, \ldots, X_p = x_p \mid Y = c_q) = \prod_{j=1}^{p} P(X_j = x_j \mid Y = c_q) \tag{4.3}$$

i.e., the independence of the p descriptors conditionally on the class.

Generally, this assumption is only an approximation, which in practice is very convenient and works well in many instances. **A key advantage conferred by the naive assumption is scalability**, as it enables estimates for each of the descriptors to be computed separately, without trying to fit all possible correlations among them (indeed, the latter becomes quickly computationally expensive with large p and requires many data points to be statistically accurate). It also allows to deal with mixed-type variable vectors seamlessly (e.g. with both continuous and discrete features).

To estimate (4.3), there are two standard options:

- For discrete variables, we can simply count the cases in the data:

$$P(X_j = x_j \mid Y = c_q) \approx \frac{\sum_{i=1}^{N} I(x_j^{(i)} = x_j \wedge y^{(i)} = c_q)}{\sum_{i=1}^{N} I(y^{(i)} = c_q)}, \quad j = 1, \ldots, p \tag{4.4}$$

  where $\wedge$ denotes the Boolean operation 'and'. In other words, $I(a \wedge b)$ will only return 1 if both a and b are true.

  Many entries might end up being empty (due to finite sampling), and so to account for this, one can use *Laplace smoothing*:

$$P(X_j = x_j \mid Y = c_q) \approx \frac{\sum_{i=1}^{N} I(x_j^{(i)} = x_j \wedge y^{(i)} = c_q) + 1}{\sum_{i=1}^{N} I(y^{(i)} = c_q) + p}$$

  This is a standard technique in statistical sampling to avoid having to deal with zero probabilities.

- Alternatively, for continuous input data, we can assume a distribution for the data and then construct estimators for the parameters of the distribution.

  A typical assumption, unsurprisingly, is that the data within each class follows a Gaussian distribution. This leads to what is called *Gaussian naive Bayes*. Specifically, assuming that each feature of the data is Gaussian in each class:

$$P(X_j = x_j \mid Y = c_q) \approx \frac{1}{\sqrt{2\pi\sigma_{j,q}^2}} e^{\frac{-(x_j - \mu_{j,q})^2}{2\sigma_{j,q}^2}}, \quad j = 1, \ldots, p \tag{4.5}$$

where we have the standard sample estimators for the mean and variance for each class q, which can be computed from our data:

$$\mu_{j,q} = \frac{1}{N_q} \sum_{y^{(i)} \in c_q} x_j^{(i)}$$

$$\sigma_{j,q}^2 = \frac{1}{N_q} \sum_{y^{(i)} \in c_q} (x_j^{(i)} - \mu_{j,q})^2$$

Here N_q is the number of samples in class q.

Normalisation factor (★): Finally, we are left with the denominator of Bayes' theorem, labelled with ★.

Using the law of total probability, we can directly rewrite the normalisation ★ in terms of our previous two estimates ♣ and ♠:

$$\begin{aligned} P(X = \mathbf{x}) &= \sum_{q=1}^{Q} P(X = \mathbf{x} \mid Y = c_q) P(Y = c_q) \\ &= \sum_{q=1}^{Q} [\clubsuit \cdot \spadesuit]_q \end{aligned}$$

This last step is just to show that our probabilities are properly normalised. Indeed, putting it all together, we end up with the following expression for the Naive Bayes classifier:

$$P(Y = c_q \mid X = \mathbf{x}) = \frac{P(Y = c_q) \prod_{j=1}^{p} P(X_j = x_j \mid Y = c_q)}{\sum_{q=1}^{Q} [\clubsuit \cdot \spadesuit]_q} = \frac{[\clubsuit \cdot \spadesuit]_q}{\sum_{q=1}^{Q} [\clubsuit \cdot \spadesuit]_q} \tag{4.6}$$

Making predictions with Naive Bayes. For a new sample $\mathbf{x}^{\text{in}}$, the Naive Bayes classifier predicts the conditional probability (4.6), thus it returns vector $\boldsymbol{\pi}^{(\text{NB})}$ of the probabilities that $\mathbf{x}^{\text{in}}$ belongs to each of the Q classes:

$$\boldsymbol{\pi}^{(\text{NB})} = \begin{bmatrix} \pi_1^{(\text{NB})} \\ \vdots \\ \pi_Q^{(\text{NB})} \end{bmatrix}, \text{ where each component is:}$$

$$\pi_q^{(\text{NB})} = P(Y = c_q \mid X = \mathbf{x}^{\text{in}}) = \frac{[\clubsuit \cdot \spadesuit]_q}{\sum_{q=1}^{Q} [\clubsuit \cdot \spadesuit]_q}$$

Clearly, we have the following property by construction:

$$\sum_{q=1}^{Q} \pi_q^{(\text{NB})} = 1$$

As in previous cases of classifiers, our output is actually a *probabilistic assignment* (soft classification). In order to get a discrete class prediction (hard classification), we need to take a discretisation, usually by taking the class with the largest probability (i.e., the argmax choice):

$$\hat{y}^{(\text{NB})} = \operatorname*{argmax}_q \boldsymbol{\pi}^{(\text{NB})}$$